PYTHON • OBJECT-ORIENTED PROGRAMMING

Abstraction in Python

Theory, real-world understanding, clean programs, outputs and interview preparation.

LEARN THE IDEA • BUILD THE CODE • EXPLAIN WITH CONFIDENCE

Prepared by

Thaneshwara M

01 The core idea

Abstraction means showing only essential functionality while hiding internal implementation details. It helps us focus on what an object does, instead of every step used to do it.

“Expose the purpose. Hide the complexity.”

WHAT users see

A simple operation such as withdraw(), pay() or start().

HOW it works internally

Validation, database calls, calculations, security checks and logging.

Real-world example: ATM

A customer inserts a card, enters a PIN, chooses Withdraw and receives cash. The customer does not need to understand bank servers, balance verification, encryption or transaction logging.

VISIBLE	HIDDEN
Insert card → Enter PIN → Withdraw	Authentication → Database → Validation → Audit

Simple definition

Abstraction creates a clear contract. The parent class says what operations are required; child classes decide how those operations are performed.

02 Why abstraction matters

Good abstraction makes software easier to understand, extend and maintain.

Simplicity

The caller uses a small, clear interface without learning complicated internal logic.

Security

Sensitive implementation details and validations remain behind controlled operations.

Consistency

Every child class follows the same required method contract.

Maintainability

Internal logic can change without changing the way callers use the object.

Extensibility

New child classes can be added with their own implementation.

Loose coupling

Programs depend on common behaviour, not one fixed implementation.

Where abstraction is commonly used

• Payment gateways • Banking applications • Notification systems
• Cloud storage • Database drivers • Vehicle and employee systems

Python support

Python uses the built-in abc module. ABC creates an Abstract Base Class and @abstractmethod marks methods that concrete child classes must implement.

```
from abc import ABC, abstractmethod
```

03 Building blocks

Abstract class

An abstract class is a blueprint that defines common structure and rules for related child classes. It may contain abstract methods, normal methods, constructors, instance variables and class variables.

Abstract method

An abstract method declares a required operation. A concrete child class must provide its implementation before objects can be created.

ABC

Base class used to declare an abstract class.

@abstractmethod

Decorator used to mark a mandatory method.

Basic syntax

```
from abc import ABC, abstractmethod

class Parent(ABC):

    @abstractmethod
    def operation(self):
        pass

class Child(Parent):

    def operation(self):
        print("Implemented by Child")
```

Rule to remember

If a child class does not implement every required abstract method, that child class also remains abstract and cannot be instantiated.

04 Program 1 • Vehicle

One common operation can have different implementations for different vehicles.

```
from abc import ABC, abstractmethod

class Vehicle(ABC):

    @abstractmethod
    def start(self):
        pass

class Car(Vehicle):

    def start(self):
        print("Car starts using a key or start button.")

class Bike(Vehicle):

    def start(self):
        print("Bike starts using a self-start button.")

car = Car()
car.start()

bike = Bike()
bike.start()
```

OUTPUT
Car starts using a key or start button.
Bike starts using a self-start button.

How it works

1. Vehicle defines the contract: every vehicle must have start().
2. Car and Bike provide their own starting logic.
3. The caller uses the same method name without knowing the internal mechanism.

05 Missing implementation

This example intentionally leaves the child class incomplete.

```
from abc import ABC, abstractmethod

class Vehicle(ABC):

    @abstractmethod
    def start(self):
        pass

class Car(Vehicle):
    pass

car = Car()
```

OUTPUT
TypeError: Can't instantiate abstract class Car
with abstract method start

Why does the error occur?

Car inherited the rule from Vehicle, but it did not implement start(). Python prevents us from creating an incomplete object.

Abstract class	Concrete class
May have incomplete behaviour	Implements all required behaviour

Correct fix

```
class Car(Vehicle):
    def start(self):
        print("Car started successfully.")
```

06 Program 2 • Bank application

An abstract class can contain a constructor, variables, normal methods and abstract methods together.

```
from abc import ABC, abstractmethod

class BankAccount(ABC):

    def __init__(self, account_number, balance):
        self.account_number = account_number
        self.balance = balance

    def show_balance(self):
        print("Account Number:", self.account_number)
        print("Balance:", self.balance)

    @abstractmethod
    def calculate_interest(self):
        pass

class SavingsAccount(BankAccount):

    def calculate_interest(self):
        interest = self.balance * 0.04
        print("Interest:", interest)

class CurrentAccount(BankAccount):

    def calculate_interest(self):
        print("Interest: 0")

savings = SavingsAccount(1001, 50000)
savings.show_balance()
savings.calculate_interest()

current = CurrentAccount(1002, 80000)
current.show_balance()
current.calculate_interest()
```

06 Bank application • Result

```
OUTPUT
Account Number: 1001
Balance: 50000
Interest: 2000.0

Account Number: 1002
Balance: 80000
Interest: 0
```

Understanding the design

The constructor and show_balance() are shared by all account types. The interest rule differs, so calculate_interest() is abstract.

Shared implementation

__init__() and show_balance() are written once in BankAccount.

Variable implementation

Each account type supplies its own calculate_interest().

What is hidden?

The caller simply invokes calculate_interest(). The caller does not need to know the percentage, formula or account-specific business rule.

Design lesson

Keep common behaviour in the abstract parent. Keep behaviour that changes in the child classes.

07 Program 3 • Multiple methods

A class may define more than one abstract method. Every concrete child must implement all of them.

```
from abc import ABC, abstractmethod

class Payment(ABC):

    @abstractmethod
    def pay(self, amount):
        pass

    @abstractmethod
    def refund(self, amount):
        pass

class UPIPayment(Payment):

    def pay(self, amount):
        print("Paid INR", amount, "using UPI")

    def refund(self, amount):
        print("Refunded INR", amount, "to UPI")

class CardPayment(Payment):

    def pay(self, amount):
        print("Paid INR", amount, "using Card")

    def refund(self, amount):
        print("Refunded INR", amount, "to Card")

payment = UPIPayment()
payment.pay(2000)
payment.refund(500)
```

OUTPUT
Paid INR 2000 using UPI
Refunded INR 500 to UPI

08 Abstraction vs Encapsulation

These ideas are related, but they solve different problems.

COMPARISON	ABSTRACTION	ENCAPSULATION
Purpose	Hides implementation complexity	Protects and groups data
Focus	What an object does	How data is controlled
Python mechanism	ABC and @abstractmethod	Classes and access conventions
Example	pay() contract	Protected account balance
Benefit	Simple, common interface	Controlled data access

Easy memory trick

Abstraction

Hide complexity. Show essential operations.

Encapsulation

Bundle and protect data inside a class.

Can both be used together?

Yes. A banking system can expose an abstract withdraw() operation while encapsulating the balance and allowing it to change only after validation.

09 Quick revision

- Abstraction hides implementation details and exposes essential functionality.
- ABC is the base used to declare an abstract class.
- @abstractmethod declares a method that child classes must implement.
- An abstract class can contain constructors, variables and normal methods.
- An incomplete abstract class cannot be instantiated.
- A concrete class implements every inherited abstract method.
- Abstraction improves simplicity, consistency, maintainability and extensibility.

Common interview questions

1. What is abstraction?

It is the process of hiding internal implementation details and showing only essential functionality.

2. How is it implemented in Python?

Using ABC and @abstractmethod from the abc module.

3. Can an abstract class have normal methods?

Yes. It can contain normal methods, abstract methods, constructors and variables.

4. Can we create an abstract-class object?

Not when it contains an unimplemented abstract method.

10 Interview-ready answer

“Abstraction is an OOP principle that hides internal implementation details and exposes only essential functionality. In Python, we implement it using the ABC class and @abstractmethod decorator from the abc module. An abstract class defines a common contract, and child classes provide the actual implementation.”

Practice task

Create an abstract class Notification with an abstract method send(message). Create EmailNotification and SMSNotification child classes and provide different implementations.

```
# Expected structure
class Notification(ABC):
    @abstractmethod
    def send(self, message):
        pass

# Create EmailNotification and SMSNotification
# Implement send() in both child classes
```

KEEP LEARNING • KEEP BUILDING

Prepared by Thaneshwara M

Python OOP Learning Series